Chapter 2 ▪ The Java Programming Environment

In this chapter, you will learn how to install the Java Development Kit (JDK) and how to compile and run Java programs. You can run the JDK tools by typing commands in a terminal window. However, many programmers prefer the comfort of an integrated development environment. You will learn how to use a freely available development environment to compile and run Java programs. Once you have mastered the techniques in this chapter and picked your development tools, you are ready to move on to **Chapter 3**, where you will begin exploring the Java programming language.

2.1. Installing the Java Development Kit

In days past, the most complete and up-to-date version of the Java Development Kit (JDK) was available from Oracle. Nowadays many different companies and organizations, including Amazon, Azul, Microsoft, and Red Hat, provide up-to-date OpenJDK builds. Each vendor has different licensing conditions and support offerings. The Eclipse Foundation provides free JDK builds for Linux, Mac OS, and Windows, which work well for learning Java.

2.1.1. Downloading the JDK

You can download the Java Development Kit from the Eclipse Foundation at **https://adoptium.net**, or from Oracle at **https://www.oracle.com/java/technologies/downloads**, or from many other providers.

Depending on the provider, the Java Development Kit may have a brand name, such as Temurin (Eclipse Foundation), Corretto (Amazon), or Zulu (Azul). The brand name is of no importance to Java programmers.

You should use the Java SE 25 (LTS) JDK. See **Table 2.1** for a summary of the acronyms and jargon that you may encounter on the download site.

Table 2.1: Java Jargon

Name	Acronym	Explanation
Java Development Kit	JDK	The software for programmers who want to write Java programs.
Java Runtime Environment	JRE	The software for running Java programs, without development tools. Only supported until Java 8. You do not want that.
Standard Edition	SE	The Java platform for use on desktops and simple server applications. You want that.
OpenJDK	—	A free and open-source implementation of Java SE.
Hotspot	—	The “just in time” compiler developed by Oracle. If asked, choose this one.
OpenJ9	—	Another “just in time” compiler developed by IBM.
GraalVM	—	An “ahead of time” compiler for executables that start quickly, but don’t support all Java features. You don’t want it for this book.
Long Term Support	LTS	A release that is supported for multiple years, unlike the six-month releases that showcase new features. Choose the latest LTS release.
Eclipse	—	A foundation that distributes open-source software.

Name	Acronym	Explanation
Temurin	—	The brand name for the OpenJDK version that the Eclipse foundation distributes.
Adoptium	—	The project within the Eclipse Foundation that provides an OpenJDK version and infrastructure support for its distribution.

2.1.2. Setting Up the JDK

After downloading the JDK, you need to install it and figure out where it was installed—you'll need that information later.

If you run Windows or have a Mac, simply launch the setup program and choose the default options.

On Linux, uncompress the `.tar.gz` file to a location of your choice, such as your home directory or `/opt`. Then set the `PATH` to the `bin` subdirectory of the directory into which the JDK was placed, such as `/opt/jdk-25.0.4/bin`. This is usually achieved by adding a line such as the following to the end of your `~/.bashrc` or `~/.bash_profile` file:

```
export PATH=/opt/jdk-25.0.4/bin:$PATH
```

Here is how you test whether you did it right. Start a terminal window. Type the line

```
javac --version
```

and press the Enter key. You should get a display such as this one:

```
javac 25.0.4
```

If instead you get a message such as “javac: command not found” or “The name specified is not recognized as an internal or external command, operable program or batch file,” then you need to double-check your installation.

It is often useful to know where the JDK is installed on your system—see **Table 2.2**. In this book, the installation directory is denoted as `$JAVA_HOME`. For example, when referring to the `$JAVA_HOME/bin` directory, I mean the directory such as `/opt/jdk-25.0.4/bin` or `C:\Program Files\Java\jdk-25\bin`. You may want to set the `JAVA_HOME` environment variable. Otherwise replace `$JAVA_HOME` in any commands with the installation directory on your system.



Caution: In the Windows terminal, use `%JAVA_HOME%` instead of `$JAVA_HOME` to refer to the `JAVA_HOME` environment variable.

Table 2.2: The JDK Installation Directory

Platform	Sample Directory
Windows (Adoptium)	`C:\Program Files\Eclipse Adoptium\jdk-25.0.4.11-hotspot`
Windows (Oracle OpenJDK)	`C:\Program Files\Java\jdk-25`
Mac OS	`/Library/Java/JavaVirtualMachines/jdk-25.0.4_11.jdk/Contents/Home`
Linux	Where you uncompressed the `.tar.gz` file, such as `/opt/jdk-25.0.4`



Note: On Windows, the Oracle JDK installer adds the directory `C:\Program Files\Common Files\Oracle\Java\javapath` to the `PATH` environment variable. That directory only contains the `javac`, `javaw`, `java`, and `jshell` executables. (You will see how to use `javac`, `java`, and `jshell` in this chapter. The `javaw` executable is a Windows-only feature for launching a program without a console window.) The other tools in the Java Development Kit can be found in the `bin` subdirectory of the JDK installation directory. When invoking those programs, either specify the complete

path (such as `C:\Program Files\Java\jdk-25\bin\javadoc` for the `javadoc` tool), or add the `bin` subdirectory to the `PATH` environment variable. One way to achieve this is with the `setx` command:

```
setx PATH "%PATH%;c:\Program Files\Java\jdk-25\bin"
```

Open another terminal window for the change to take effect.

The **https://adoptium.net** installer does not have this issue. It adds all JDK tools to the `PATH`.

2.1.3. Source Files and Documentation

To effectively work with this book, you should install the *Core Java* program examples. Download them from **https://horstmann.com/corejava**. The programs are packaged into a zip file `corejava.zip`. Just unzip them into any directory of your choice. The files are located in two subdirectories `corejava-bookcode` and `corejava-modulechapter`. In the following examples, they are placed into the home directory.

The library source files are delivered in the JDK as a compressed file `$JAVA_HOME/lib/src.zip`. If you are interested in learning how the Java API is implemented, unzip that file to get access to the source code.



Tip: The `src.zip` file contains the source code for all public libraries. To obtain even more source (for the compiler, the virtual machine, the native methods, and the private helper classes), go to **https://openjdk.org**.

You can read the JDK documentation at **https://docs.oracle.com/en/java/javase/25/docs/**. If you have a nomadic lifestyle and prefer an offline version, visit **https://www.oracle.com/java/technologies/downloads** and look for a "Documentation Download" button or link.



Tip: With many browsers, you can define your own search engine. It is very handy to set up search for the Java API. The search string is `https://docs.oracle.com/en/java/javase/21/docs/api/search.html?`

`q=%s`. If you associate this with a key such as `@j`, then you can type queries like `@j println` into your browser's address bar.

2.2. Using the Command-Line Tools

If your programming experience comes from a development environment such as Microsoft Visual Studio, you are accustomed to a system with a built-in text editor, menus to compile and launch a program, and a debugger. The JDK contains nothing even remotely similar. You do *everything* by typing in commands in a terminal window. This may sound cumbersome, but it is nevertheless an essential skill. When you first install Java, you will want to troubleshoot your installation before you install a development environment. Moreover, by executing the basic steps yourself, you gain a better understanding of what a development environment does behind your back.

However, after you have mastered the basic steps of compiling and running Java programs, you will want to use a professional development environment. You will see how to do that in the following section.

Let's get started the hard way: launching a Java program from the command line.

1. Open a terminal window.
2. Change to the `corejava-bookcode` directory (where you installed the source code for the book examples, as explained in **Section 2.1.3**.)
3. Enter the following command:

```
java Welcome.java
```

In the terminal window, you should see the output shown in **Figure 2.1**.



Figure 2.1: Running the `Welcome.java` program

Congratulations! You have just run your first Java program.



Note: In prior versions of Java, you had to use two steps:

```
javac Welcome.java
java Welcome
```

The `javac` program is the Java compiler. It compiles the file `Welcome.java` into the file `Welcome.class`. The `java` program launches the Java virtual machine. It executes the bytecodes that the compiler placed in the class file. Note that here we launch the `java` program with the name of the class, not a file name. Then the `java` program simply loads the class file and runs it.

It is more efficient to compile a program once and then run it whenever you need it, by calling `javac` and `java` separately. But for simple demo programs, just run `java` followed by the file name, to compile and run the program.

The `Welcome` program is extremely simple. It merely prints a message to the terminal. You may enjoy looking inside the program, shown in **Listing 2.1**. You will see how it works in the next chapter.



Note: This program uses Java 25 features that were designed to make simple programs shorter. If you use an older version of Java, download the sample code of the preceding edition which is available at **https://horstmann.com/corejava/corejava13.zip**.

Listing 2.1 Welcome.java

```
1  /**
2   * This program displays a greeting for the reader.
3   */
4  void main() {
5      String greeting = "Welcome to Core Java!";
6      IO.println(greeting);
7      IO.println("=".repeat(greeting.length()));
8  }
```

In the age of integrated development environments, many programmers are unfamiliar with running programs in a terminal window. Any number of things can go wrong, leading to frustrating results.

Pay attention to the following points:

1. If you type in the program by hand, make sure you correctly enter the uppercase and lowercase letters. For example, you must type `main` and not `Main` or `MAIN`.
2. If you get a message such as “Bad command or file name” or “javac: command not found,” go back and double-check your installation, in particular the executable path setting.
3. If `java` or `javac` reports that it cannot find the file `Welcome.java`, you should check whether that file is present in the directory.
   Under Linux, check that you used the correct capitalization for `Welcome.java`.

Some Windows text editors insist on adding an extension `.txt` to every file's name. Under the default settings, Windows Explorer hides the `.txt` extension because it belongs to a "known file type." Therefore, use the `dir` console command, *not* the graphical Explorer tool, to check the actual file name. Rename it if necessary.

The `Welcome` program was not terribly exciting. Next, try out a graphical application. This program is a simple image file viewer that loads and displays an image. As before, run the program from the command line.

1. Open a terminal window.
2. Change to the directory `corejava-bookcode`.
3. Enter the following:

```
java ImageViewer.java
```

A new program window pops up with the ImageViewer application. Now, select File → Open and look for an image file to open. (There are sample files in the `images` directory.) The image is displayed (see **Figure 2.2**). To close the program, click on the Close box in the title bar or select File → Exit from the menu.



Figure 2.2: Running the ImageViewer application

Have a quick look at the source code (**Listing 2.2**). The program is substantially longer than the first program, but it is not too complex if you consider how much code it would take in C or C++ to write a similar application. Of course, nowadays it is not common to write desktop applications with graphical user interfaces, but if you are interested, you can find more details in **Chapter 10 of Volume II**.

Listing 2.2 ImageViewer.java

```
import module java.desktop;

/**
 * A program for viewing images.
 */
void main() {
    EventQueue.invokeLater(() -> {
        var frame = new ImageViewerFrame();
        frame.setTitle("ImageViewer");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    });
}
```

```
14
15  /**
16   * A frame with a label to show an image.
17   */
18  class ImageViewerFrame extends JFrame {
19      private static final int DEFAULT_WIDTH = 300;
20      private static final int DEFAULT_HEIGHT = 400;
21
22      public ImageViewerFrame() {
23          setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);
24
25          // use a label to display the images
26          var label = new JLabel();
27          add(label);
28
29          // set up the file chooser
30          var chooser = new JFileChooser();
31          chooser.setCurrentDirectory(new File("images"));
32
33          // set up the menu bar
34          var menuBar = new JMenuBar();
35          setJMenuBar(menuBar);
36
37          var menu = new JMenu("File");
38          menuBar.add(menu);
39
40          var openItem = new JMenuItem("Open");
41          menu.add(openItem);
42          openItem.addActionListener(_ -> {
43              // show file chooser dialog
44              int result = chooser.showOpenDialog(null);
45
46              // if file selected, set it as icon of the label
47              if (result == JFileChooser.APPROVE_OPTION) {
48                  String name = chooser.getSelectedFile().getPath();
49                  label.setIcon(new ImageIcon(name));
50              }
51          });
52
53          var exitItem = new JMenuItem("Exit");
54          menu.add(exitItem);
```

```
55          exitItem.addActionListener(_ -> System.exit(0));
56      }
57  }
```

2.3. Using an Integrated Development Environment

In the preceding section, you saw how to compile and run a Java program from the command line. That is a useful skill for troubleshooting, but for most day-to-day work, you should use an integrated development environment. These environments are so powerful and convenient that it simply doesn't make much sense to labor on without them. Excellent choices are the freely available Eclipse, IntelliJ IDEA, NetBeans, and Visual Studio Code. In this section, you will see how to get started with Eclipse. Of course, if you prefer a different development environment, you can certainly use it with this book.

Get started by downloading Eclipse from **https://eclipse.org/downloads**. Versions exist for Linux, Mac OS X, and Windows. Run the installation program and pick the installation set called "Eclipse IDE for Java Developers."

Here are the steps to load the sample code into Eclipse:

1. After starting Eclipse, select File → New → Project from the menu.
2. Select "Java Project" from the wizard dialog (see **Figure 2.3**).

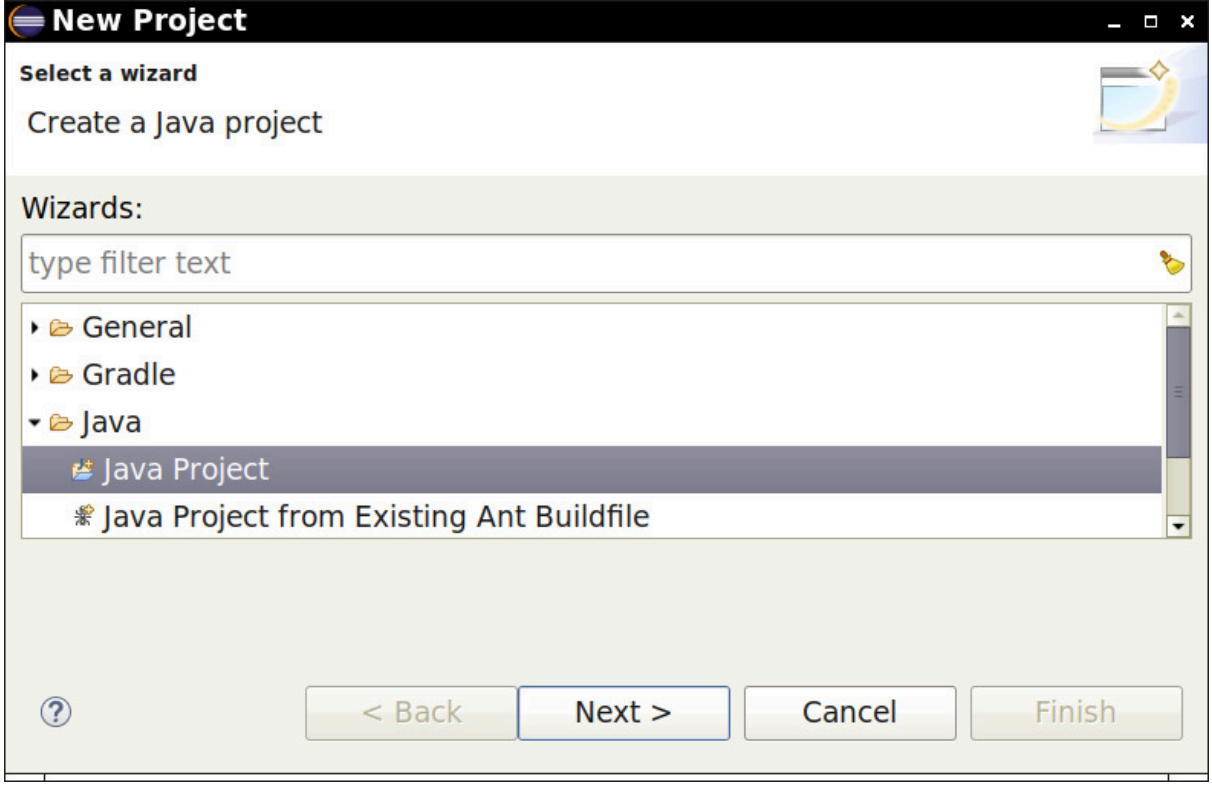


Figure 2.3: The New Project dialog in Eclipse

3. Click the Next button. *Uncheck* the "Use default location" checkbox. Click on Browse and navigate to the `corejava-bookcode` directory (**Figure 2.4**).

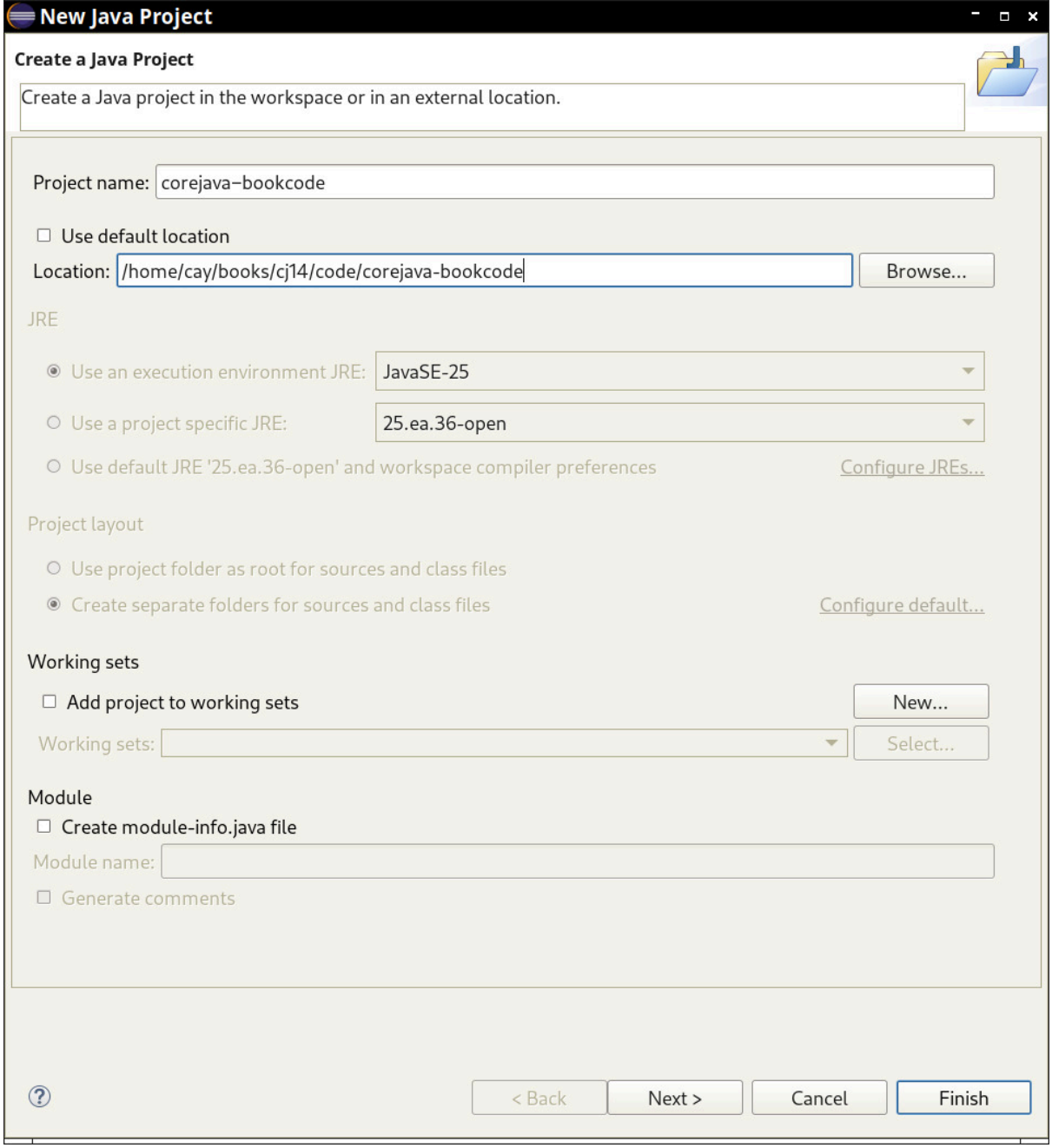


Figure 2.4: Configuring a project in Eclipse

4. Uncheck the box labeled “Create modul-info.java file.” Click the Finish button. The project is now created.
5. Click on the triangles in the left pane next to the project until you locate the file `Welcome.java` inside the “default package”, and double-click on it. You should now see a pane with the program code.
6. With the right mouse button, click on the file name (Welcome.java) in the left pane. Select Run → Run As → Java Application. The program output is displayed in the console pane (see **Figure 2.5**).

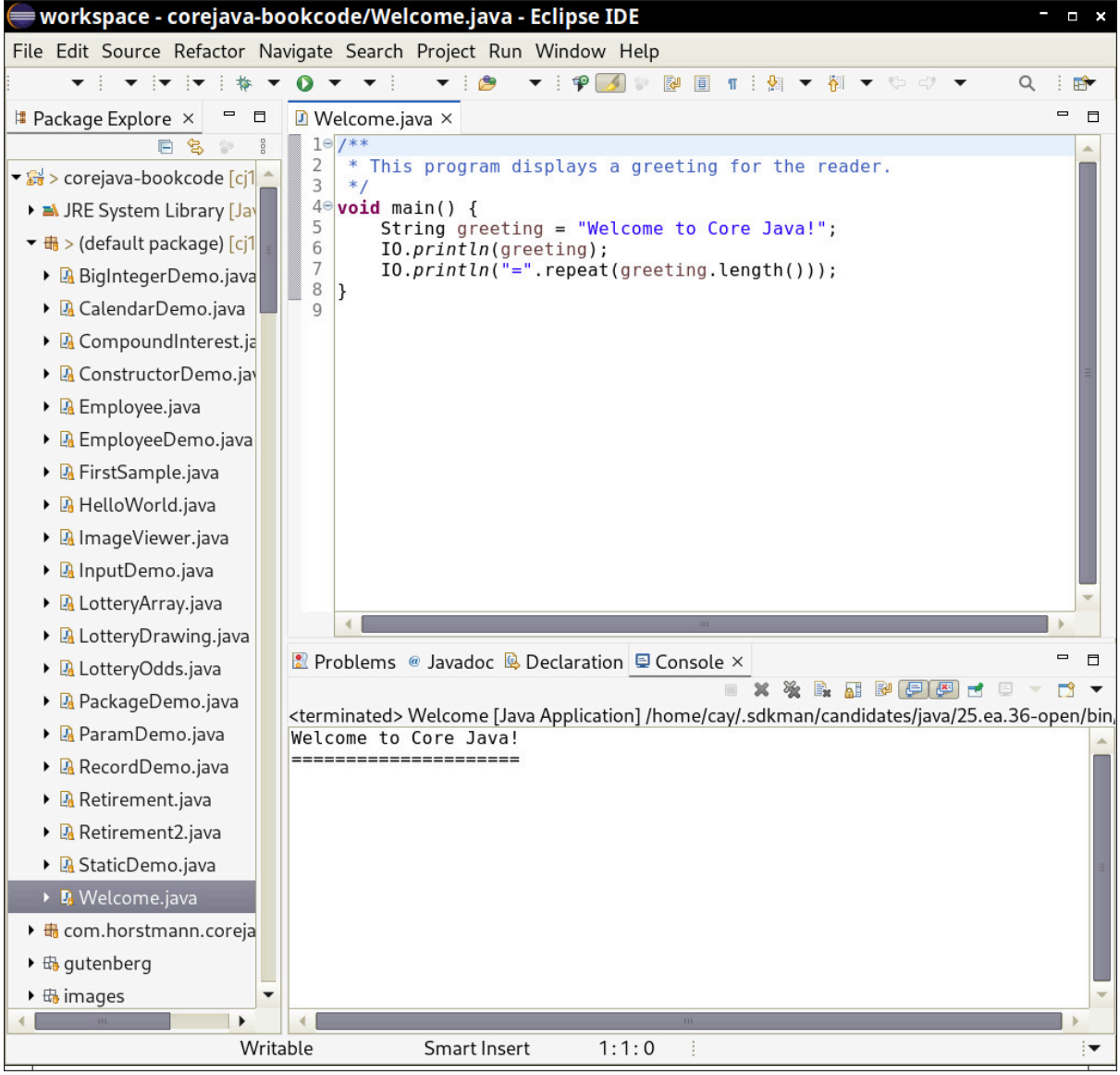


Figure 2.5: Running a source file with Eclipse

2.4. JShell

The JShell program provides a “read-evaluate-print loop,” or REPL. You type a Java expression; JShell evaluates your input, prints the result, and waits for your next input. This is an excellent way to experiment—much faster than writing a complete program in an integrated development environment.

To start JShell, simply type `jshell` in a terminal window (see **Figure 2.6**).

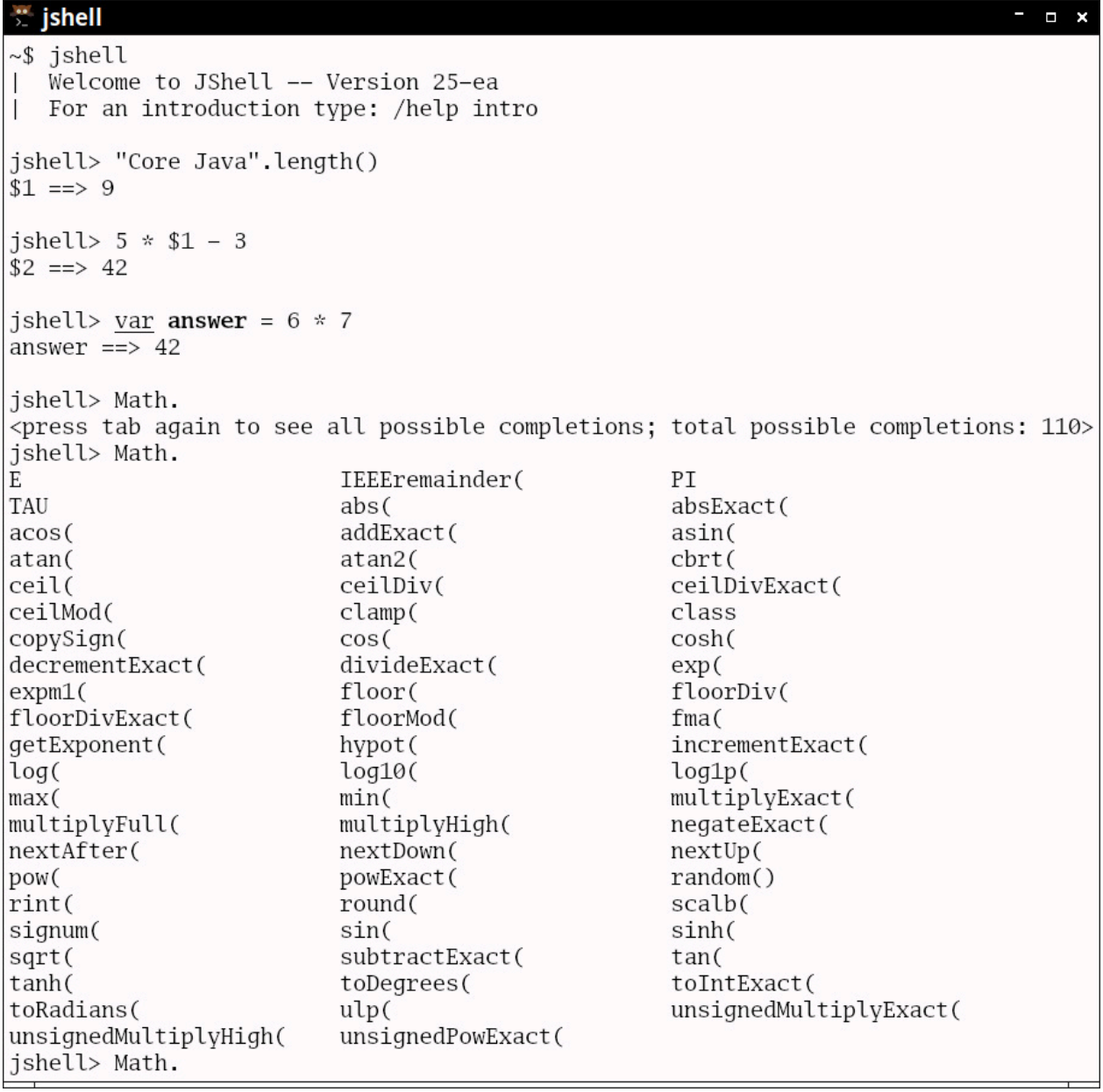


Figure 2.6: Running JShell

JShell starts with a greeting, followed by a prompt:

```
|  Welcome to JShell -- Version 25.0.4
|  For an introduction type: /help intro

jshell>
```

Now type an expression, such as

```
"Core Java".length()
```

JShell responds with the result—in this case, the number of characters in the string “Core Java”.

```
$1 ==> 9
```

Note that you do *not* type `IO.println`. JShell automatically prints the value of every expression that you enter.

The `$1` in the output indicates that the result is available in further calculations. For example, if you type

```
5 * $1 - 3
```

the response is

```
$2 ==> 42
```

If you need a variable many times, you can give it a more memorable name. For example,

```
jshell> var answer = 6 * 7
answer ==> 42
```

Another useful feature is tab completion. Type

```
Math.
```

followed by the Tab key. Because there are so many completions, you are prompted to hit the Tab key again. You get a list of all methods that you can invoke with the `Math` class:

```
jshell> Math.
E                       IEEEremainder(          PI
TAU                     abs(                    absExact(
```

```
acos(                   addExact(               asin(
atan(                   atan2(                  cbrt(
ceil(                   ceilDiv(                ceilDivExact(
ceilMod(                clamp(                  class
copySign(               cos(                    cosh(
decrementExact(         divideExact(            exp(
expm1(                  floor(                  floorDiv(
floorDivExact(          floorMod(               fma(
getExponent(            hypot(                  incrementExact(
log(                    log10(                  log1p(
max(                    min(                    multiplyExact(
multiplyFull(           multiplyHigh(           negateExact(
nextAfter(              nextDown(               nextUp(
pow(                    random()                rint(
round(                  scalb(                  signum(
sin(                    sinh(                   sqrt(
subtractExact(          tan(                    tanh(
toDegrees(              toIntExact(             toRadians(
ulp(                    unsignedMultiplyHigh(
```

Now type `l` and hit the Tab key again. The method name is completed to `log`, and you get a shorter list:

```
jshell> Math.log
log(     log10(   log1p(
```

Now you can fill in the rest by hand:

```
jshell> Math.log10(0.001)
$4 ==> -3.0
```

To repeat a command, hit the ↑ key until you see the line that you want to reissue or edit. You can move the cursor in the line with the ← and → keys, and add or delete characters. Hit Enter when you are done. For example, hit the ↑ key and replace `0.001` with `1000`, then hit Enter:

```
jshell> Math.log10(1000)
$5 ==> 3.0
```

You exit JShell with the command

```
/exit
```

JShell makes it easy and fun to learn the Java language and library without having to launch a heavy-duty development environment and without fussing with a program and a `main` method.

In this chapter, you learned about the mechanics of compiling and running Java programs. You are now ready to move on to **Chapter 3** where you will start learning the Java language.

Previous chapter
1. An Introduction to Java

Next chapter
3. Fundamental Programming Structures in Java